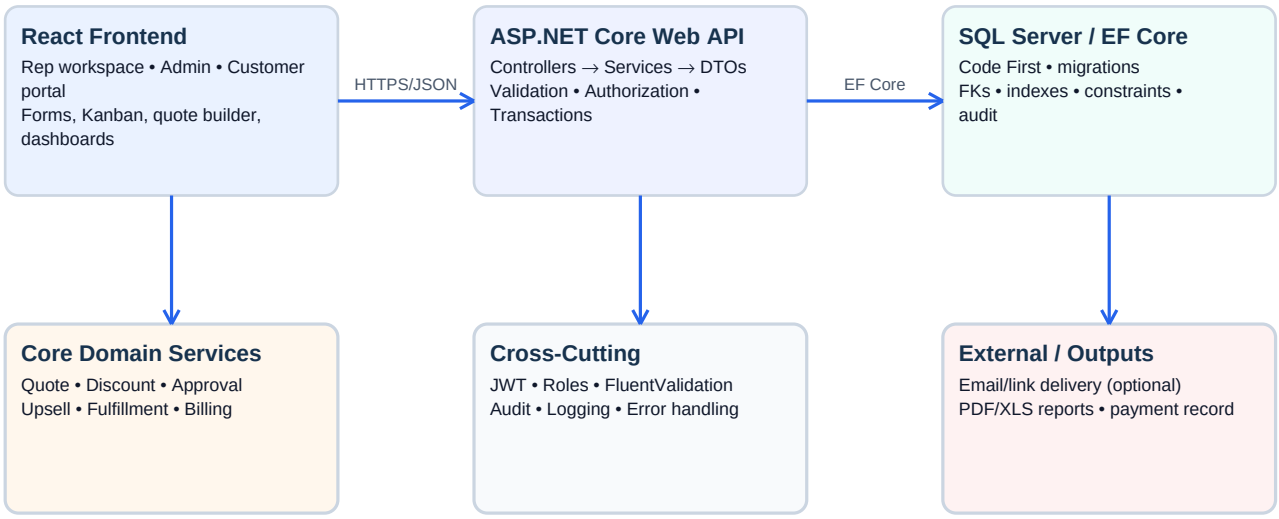
DealFlow360

Complete 24-Hour Hackathon Implementation Specification

Chosen stack: ASP.NET Core Web API • Entity Framework Core Code First • SQL Server • React



One application • one source of truth • all business rules enforced server-side

Purpose. This document converts the supplied DealFlow360 problem statement into an implementation-ready design: roles, tables, endpoints, DTOs, validations, service logic, status transitions, authorization, frontend routes, testing, seed data, and a 24-hour build order.

Important source boundary. The challenge document permits any language/framework/database and explicitly prioritizes business logic, data modeling, and end-to-end workflow. The exact endpoint names, table names, DTO names, and internal implementation below are therefore our engineering design—not wording mandated by the challenge.

Source: supplied DealFlow360 problem statement, 13 pages. Core requirements include discount governance, upsell/cross-sell, multi-warehouse fulfillment, hybrid billing, deal health/anomalies, customer portal negotiation, backend configuration, reporting, and an end-to-end quotation-to-cash flow.

1. What We Are Actually Building

DealFlow360 is not just a CRM, quote form, or inventory screen. It is a **self-governing sales transaction engine**. The system must watch the quotation as it is built, enforce pricing rules, route risky discounts to the right people, react to warehouse stock, support one-time and recurring lines together, allow the customer to negotiate through a restricted portal, and expose deal-risk/reporting information.

The supplied statement explicitly calls out multi-tier discount governance, live upsell/cross-sell suggestions, multi-warehouse fulfillment and backorders, hybrid billing, deal health/anomaly alerts, customer portal negotiation, and sales backend configuration/reporting. ■

The key design principle is: **the frontend asks; the backend decides.** React may display a calculated result, but the ASP.NET service layer is the authority for discount approval, totals, margin, warehouse allocation, billing schedules, negotiation re-approval, and permissions.

1.1 The business lifecycle

Customer/quotation → product selection → price list → discount validation → blended risk → approval (if needed) → upsell/cross-sell → customer negotiation → re-approval if terms worsen → order confirmation → warehouse allocation → fulfillment/backorder → one-time billing + recurring billing schedule → payment → deal health/reporting.

1.2 What counts as 'done'

- Internal users can sign up/log in; role permissions actually restrict endpoints.
- Admin can configure products, price lists, discount tiers, approval chain, warehouses, subscription plans, and optional upsell rules.
- Rep can create/edit a quotation and immediately see totals, margin, discount risk, and suggestions.
- Discount governance is calculated from customer tier + category limits; mixed-category quotes get a blended risk result.
- Approval is automatic; the rep does not manually choose a manager.
- Approved quotes can be converted to an order and warehouse stock can be split across warehouses.
- An order can contain one-time products and recurring subscription lines; recurring lines generate billing schedule records.
- Customer portal is a separate restricted surface; customer requests can change quote state and trigger re-approval.
- Deal-health and anomaly rules are computed from actual data, not mocked UI labels.
- A payment updates invoice/payment status correctly.
- Reporting filters work by period, sales rep/team, approval status, and product/category.

2. Roles, Users, and Authorization Model

The source defines five practical roles: Sales Rep, Sales Manager/Approver, Finance/Operations User, Customer (Portal User), and Admin. The role responsibilities are stated on pages 2–3 of the supplied PDF. ■

Role	Primary responsibility	Main screens	API access
Admin	Backend setup + platform analytics	Admin config, reports	All configuration + read/report; user/role administration
SalesRep	Build quotes, discounts, upsell, track approval/fulfillment, respond to negotiation	Workspace, quotes, pipeline	Own quotes/deals + operational read/actions
SalesManager	Approve/reject/return risky quotes; configure discount tiers/chains; monitor health	Approval queue, health dashboard	Approval endpoints + assigned/team visibility
FinanceOperations	Second-level approval, warehouse splits/backorders, billing reconciliation	Fulfillment, billing	Finance approval + fulfillment + billing
Customer	View/negotiate/confirm own quotation	Portal only	Only own portal quotation/token resources

2.1 Authentication

Use JWT bearer authentication for internal APIs. Store only password hashes, never passwords. Customer portal access must be restricted separately from internal APIs; a customer token/session must only resolve to its own quotation/customer record.

Recommended internal token claims: **sub** = UserId, **role** = role name, **teamId** = team/team scope, **email**, **jti** for audit/trace if desired.

2.2 Authorization rule

Use policy/role authorization in ASP.NET. Do not rely on React hiding a button. Every protected endpoint checks authentication and role, and services additionally check ownership/record scope. Example: a SalesRep can update only quotations they own; a Customer can only read/negotiate the quotation linked to their portal identity.

2.3 Signup rule for a hackathon

The statement says internal users can sign up and log in. For a safe demo, public signup may create only a SalesRep role. Admin/Manager/Finance roles should be seed-created or assigned by Admin. Customer users should be created through customer/portal setup, not by public internal signup.

3. Database Design — SQL Server + EF Core Code First

The database should be relational because quotations, lines, approvals, warehouse allocations, billing schedules, payments, and audit records have strong relationships and transactional rules.

3.1 Recommended tables

#	Table	Purpose	Key relationships
1	Users	Internal and portal identities	Role; Customer optional
2	Roles	Role catalog	Users
3	Customers	B2B customer master + tier	Quotes, Orders, Invoices
4	CustomerTiers	Bronze/Silver/Gold and limits	Customers, discount rules
5	SalesTeams	Rep/team reporting scope	Users
6	Products	Product/service/subscription master	Categories, price lists, quote lines
7	ProductCategories	Hardware/Service/etc.	Products, discount rules
8	ProductVariants	Optional attributes/values/extra price	Products
9	PriceLists	Named pricing schemes	PriceListItems
10	PriceListItems	Customer tier/currency/product pricing	PriceLists, Products
11	DiscountRules	Tier/category ceilings and risk bands	CustomerTiers, Categories
12	ApprovalRules	Maps risk/discount range to approval level/order	Approval chain
13	Warehouses	Warehouse master + shipping weighting	Stock, allocations
14	InventoryStocks	Per warehouse/product quantity	Warehouses, Products
15	ReplenishmentRules	Optional low-stock settings	Warehouse/Product
16	SubscriptionPlans	Monthly/quarterly/yearly plans + proration rules	Products
17	UpsellCrossSellRules	Product pairings/promotions/margin threshold	Products
18	Quotations	Header, customer, owner, totals, status, risk	Lines, approvals, negotiation
19	QuotationLines	Products, qty, price, discount, margin	Quotation, Product
20	QuotationLineComments	Customer line questions/requests	QuotationLine, Customer/User
21	QuotationChanges	Negotiation/change history	Quotation
22	ApprovalRequests	Approval workflow instances	Quotation
23	ApprovalActions	Approve/reject/return audit per step	ApprovalRequest, User
24	Orders	Confirmed commercial order	Quotation, Fulfillment, Billing
25	OrderLines	Snapshot of confirmed lines	Order, Product
26	WarehouseAllocations	Split order quantities by warehouse	OrderLine, Warehouse
27	Backorders	Unfulfilled quantities	Order/warehouse
28	BillingSchedules	Recurring billing schedule instances	OrderLine, SubscriptionPlan
29	Invoices	One-time/recurring invoice records	Order, Customer
30	InvoiceLines	Invoice line snapshots	Invoice, Product
31	Payments	Payment records	Invoice
32	CreditNotes	Partial refund/credit note triggers	Invoice, OrderLine
33	DealHealthSnapshots	Computed health/anomaly results	Quotation/Order
34	AuditLogs	Who changed what and why	User, entity
35	Notifications	In-app workflow notifications	User
36	RefreshTokens	Optional secure JWT refresh	User

35–36 tables are the full production-style design. For a 24-hour hackathon, do not build every optional table first. The **minimum core** is Users, Roles, Customers, CustomerTiers, SalesTeams, Products, Categories, PriceLists, PriceListItems, DiscountRules, ApprovalRules, Warehouses, InventoryStocks, SubscriptionPlans, Quotations, QuotationLines, ApprovalRequests, ApprovalActions, Orders, OrderLines, WarehouseAllocations, Backorders, BillingSchedules, Invoices, InvoiceLines, Payments,

DealHealthSnapshots, AuditLogs. Upsell rules, variants, notifications, credit notes, and refresh tokens can be simplified or added after the core flow.

4. Entity Rules and SQL/EF Core Conventions

Use integer/long identity keys for simplicity. Use decimal(18,2) for money and decimal(5,2) for percentages. Store dates in UTC. Use row timestamps such as CreatedAtUtc and UpdatedAtUtc. Use explicit foreign keys and indexes.

Entity	Important fields	Rules
User	Id, Name, Email, PasswordHash, RoleId, SalesTeamId, CustomerId?, IsActive	Unique normalized email; inactive users cannot log in
Customer	Id, Name, Email, Phone, TierId, CurrencyCode, IsActive	Tier required for discount governance
Product	Id, SKU, Name, CategoryId, ProductType, BasePrice, CostPrice, TaxRate, Unit, IsActive	SKU unique; BasePrice >= 0; CostPrice >= 0
PriceListItem	PriceListId, ProductId, CurrencyCode, UnitPrice	Unique (PriceList, Product, Currency)
DiscountRule	TierId, CategoryId?, MaxDiscountPercent, ManagerThreshold, FinanceThreshold	0 <= limits <= 100
ApprovalRule	Level, MinRisk, MaxRisk, Role, Sequence	Sequence controls Manager -> Finance
Warehouse	Id, Name, ShippingCostWeight, IsActive	Weight >= 0
InventoryStock	WarehouseId, ProductId, OnHand, Reserved	Unique warehouse/product; available = onHand - reserved
Quotation	Number, CustomerId, SalesRepId, Status, Total, DiscountTotal, MarginTotal, RiskScore, ApprovalStatus	Number unique; totals server-calculated
QuotationLine	QuotationId, ProductId, Quantity, UnitPrice, DiscountPercent, NetAmount, MarginAmount	Quantity > 0; discount 0..100
ApprovalRequest	QuotationId, Level, Status, Sequence, RequestedAt, ActedAt, Reason	Only current sequence can act
Order	Number, QuotationId, CustomerId, Status, Total	Created only from eligible confirmed quote
WarehouseAllocation	OrderLineId, WarehouseId, Quantity, ShipmentCost	Quantity > 0; never exceed available stock
BillingSchedule	OrderLineId, PlanId, StartDate, EndDate, NextBillingDate, Quantity, UnitPrice, Status	Recurring line only
Invoice	Number, OrderId, Type, Status, Total, PaidAmount, DueDate	PaidAmount <= Total
Payment	InvoiceId, Amount, PaidAtUtc, Reference	Amount > 0; cumulative payments <= invoice total
AuditLog	UserId, EntityName, EntityId, Action, OldValueJson, NewValueJson, Reason, CreatedAtUtc	Append-only

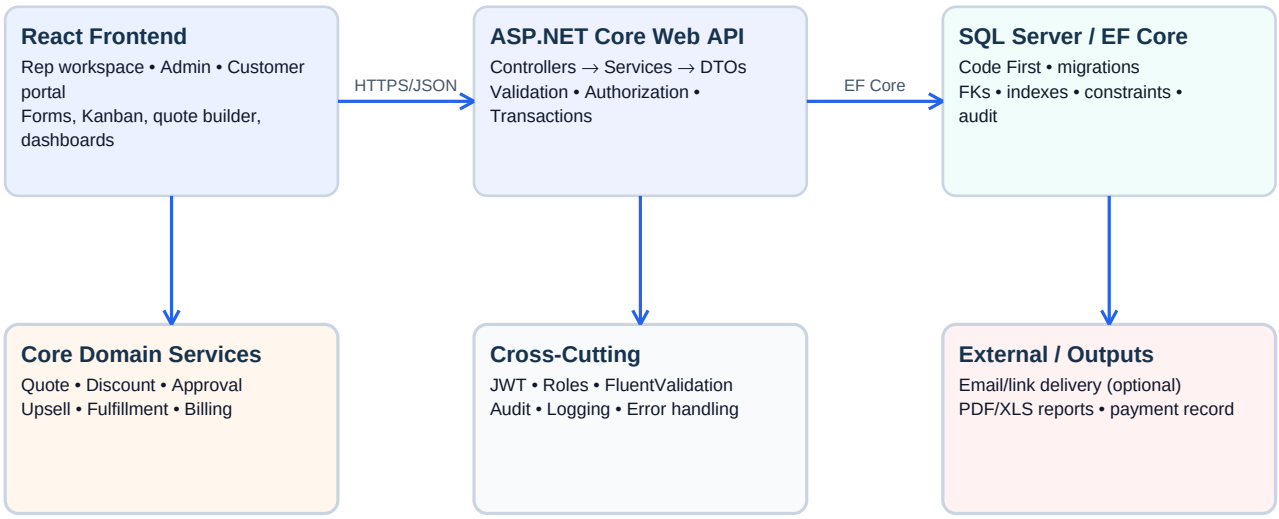
4.1 EF Core relationships

- Customer 1→many Quotations, Orders, Invoices; Customer many→1 CustomerTier.
- Quotation 1→many QuotationLines; Quotation 1→many ApprovalRequests and QuotationChanges.
- QuotationLine many→1 Product; Product many→1 Category.
- Order 1→many OrderLines; OrderLine 1→many WarehouseAllocations and optionally BillingSchedules.
- Invoice 1→many InvoiceLines and Payments.
- Warehouse 1→many InventoryStocks; Product 1→many InventoryStocks.
- AuditLog references an entity by EntityName + EntityId to keep auditing generic.

4.2 Do not store derived values blindly

Quotation Total, margin, risk score, available stock, invoice outstanding amount, and deal-health status should be recalculated by services from authoritative records. Cached snapshots are acceptable for dashboards, but the transactional result must come from server-side logic.

5. ASP.NET Core Architecture



One application • one source of truth • all business rules enforced server-side

5.1 Recommended solution structure

```
DealFlow360.sln
■■■ DealFlow360.Api/
■ ■■■ Controllers/
■ ■■■ Middleware/
■ ■■■ Extensions/
■ ■■■ Program.cs
■■■ DealFlow360.Application/
■ ■■■ DTOs/
■ ■■■ Interfaces/
■ ■■■ Services/
■ ■■■ Validators/
■ ■■■ Policies/
■■■ DealFlow360.Domain/
■ ■■■ Entities/
■ ■■■ Enums/
■ ■■■ Rules/
■■■ DealFlow360.Infrastructure/
■■■ Data/
■■■ Configurations/
■■■ Migrations/
■■■ Repositories/ (only if genuinely needed)
```

Do not create repositories for every table just because a pattern says so. For a 24-hour project, EF Core DbContext + focused domain services is simpler and faster. Controllers should be thin; all business decisions belong in services.

5.2 Service boundaries

Service	Owns
AuthService	Signup/login/password hashing/token creation
CustomerService	Customer/tier reads and management
ProductService	Products/categories/variants
PriceListService	Price selection by customer tier/currency
DiscountService	Per-line limits + blended risk score
ApprovalService	Approval route, approve/reject/return, re-entry

Service	Owns
QuotationService	Quote lifecycle, totals, status transitions
RecommendationService	Upsell/cross-sell ranking + margin delta
FulfillmentService	Warehouse split, reserve, backorder, consolidate
BillingService	Invoice creation, recurring schedules, proration/credits
NegotiationService	Portal changes, counters, re-approval
DealHealthService	Stall, discount anomaly, delivery slippage
ReportingService	Dashboard KPIs and filtered report exports
AuditService	Append-only audit events
NotificationService	In-app/email-ready notifications

5.3 Transaction boundary

Any command that changes several related records must be transactional. Examples: approve quotation + create audit + advance status; confirm order + reserve stock + create allocations; record payment + update invoice status. Use EF Core Database.BeginTransactionAsync or TransactionScope where appropriate.

6. REST API Conventions

Base URL: **/api**. Use plural nouns for resources and action endpoints only when an operation is not naturally CRUD. JSON over HTTPS. All response objects use a predictable envelope where practical.

6.1 Standard response shapes

```
Success: { "success": true, "data": { ... }, "message": "..." }
Validation error: { "success": false, "message": "Validation failed", "errors": { "field":
["reason"] }, "traceId": "..." }
Business rule error: { "success": false, "message": "Quotation requires manager approval",
"code": "APPROVAL_REQUIRED", "traceId": "..." }
```

HTTP	Use
200	Successful GET/update/calculation
201	Created resource
204	Successful delete where no body is needed
400	Malformed request / validation / invalid state transition
401	Missing/invalid authentication
403	Authenticated but not authorized
404	Resource does not exist or is outside caller scope
409	Concurrency/conflict or duplicate/business conflict
422	Optional semantic validation for complex business input
500	Unexpected server error; never expose stack trace

6.2 DTO principle

Never bind EF entities directly from the frontend. Create request DTOs for commands and response DTOs for reads. This prevents over-posting and keeps API contracts stable.

7. API Catalogue — Authentication & Users

Method	Route	Auth	Meaning
POST	/api/auth/signup	Anonymous	Create internal SalesRep account
POST	/api/auth/login	Anonymous	Authenticate and return JWT
POST	/api/auth/refresh	Anonymous/Refresh token	Issue new access token if refresh tokens are used
POST	/api/auth/logout	Authenticated	Invalidate refresh token/session if implemented
GET	/api/auth/me	Any internal	Return current user and role
GET	/api/users	Admin	List users
GET	/api/users/{id}	Admin/Manager	View user
POST	/api/users	Admin	Create internal user with controlled role
PUT	/api/users/{id}	Admin	Update user/role/team/active state
PATCH	/api/users/{id}/status	Admin	Activate/deactivate

DTOs

DTO	Fields
SignupRequest	fullName, email, password, confirmPassword
LoginRequest	email, password
AuthResponse	accessToken, expiresAtUtc, user { id, name, email, role, teamId }
UpdateUserRequest	fullName, roleId, salesTeamId, isActive
MeResponse	id, name, email, role, team, permissions

Validation

- Email required, normalized to lowercase, unique.
- Password minimum 8 characters; require upper/lower/digit for a strong hackathon implementation.
- confirmPassword must match.
- Inactive user cannot log in.
- Only Admin may assign Admin/Manager/Finance roles.
- JWT must include role; never trust a role sent in request body.

Route-level authorization

Role	Auth endpoints
Anonymous	signup, login
Internal users	me, logout
Admin	all user management
Customer	none of internal auth endpoints; portal auth is separate

8. API Catalogue — Backend Configuration

These APIs correspond directly to the Admin configuration area described in the source: products/price lists, discount tiers/approval chains, warehouses/stock/replenishment, subscription plans, optional upsell rules, and reporting configuration.

Method	Route	Role	Purpose
GET	/api/customer-tiers	Admin, Manager, SalesRep	List customer tiers
POST	/api/customer-tiers	Admin	Create tier
PUT	/api/customer-tiers/{id}	Admin	Update tier
GET	/api/categories	Authenticated internal	List categories
POST	/api/categories	Admin	Create category
PUT	/api/categories/{id}	Admin	Update category
GET	/api/products	Authenticated internal	Search/list products
POST	/api/products	Admin	Create product
GET	/api/products/{id}	Authenticated internal	Product detail
PUT	/api/products/{id}	Admin	Update product
GET	/api/products/{id}/variants	Authenticated internal	List variants
POST	/api/products/{id}/variants	Admin	Create variant
GET	/api/price-lists	Admin, SalesRep, Manager	List price lists
POST	/api/price-lists	Admin	Create price list
PUT	/api/price-lists/{id}	Admin	Update price list
POST	/api/price-lists/{id}/items	Admin	Add product price
PUT	/api/price-lists/{id}/items/{itemId}	Admin	Update product price
DELETE	/api/price-lists/{id}/items/{itemId}	Admin	Remove product price
GET	/api/discount-rules	Admin, Manager	List discount rules
POST	/api/discount-rules	Admin, Manager	Create discount rule
PUT	/api/discount-rules/{id}	Admin, Manager	Update discount rule
GET	/api/approval-rules	Admin, Manager	List approval routing rules
POST	/api/approval-rules	Admin, Manager	Create approval rule
PUT	/api/approval-rules/{id}	Admin, Manager	Update approval rule
GET	/api/warehouses	Authenticated internal	List warehouses
POST	/api/warehouses	Admin, FinanceOperations	Create warehouse
PUT	/api/warehouses/{id}	Admin, FinanceOperations	Update warehouse
GET	/api/warehouses/{id}/stock	Authenticated internal	View stock
PUT	/api/warehouses/{id}/stock/{productId}	Admin, FinanceOperations	Adjust stock
GET	/api/subscription-plans	Authenticated internal	List plans
POST	/api/subscription-plans	Admin	Create plan
PUT	/api/subscription-plans/{id}	Admin	Update plan
GET	/api/upsell-rules	Admin, Manager	List suggestion rules
POST	/api/upsell-rules	Admin	Create pairing/promotion rule
PUT	/api/upsell-rules/{id}	Admin	Update rule

9. API Catalogue — Customers, Sales Teams, Pipeline

Method	Route	Role	Purpose
GET	/api/customers?search=&tierId;=	Internal	Search customers
POST	/api/customers	Admin, SalesRep, Manager	Create customer
GET	/api/customers/{id}	Internal	Customer detail
PUT	/api/customers/{id}	Admin, SalesRep, Manager	Update customer
GET	/api/customers/{id}/quotations	Internal	Customer quote history
GET	/api/customers/{id}/orders	Internal	Customer order history
GET	/api/sales-teams	Admin, Manager	List teams
POST	/api/sales-teams	Admin	Create team
PUT	/api/sales-teams/{id}	Admin	Update team
GET	/api/pipeline	SalesRep, Manager, Admin	Kanban/pipeline data

Pipeline response should be grouped by quotation stage/status and include only summary data: quotation id/number, customer, amount, owner, stage, approval status, health status, last activity, expected close date.

10. API Catalogue — Quotation Builder (Core)

This is the most important API group. The frontend should never calculate authoritative totals itself. It can calculate optimistically for UX, but every save/confirm/reprice operation calls the backend.

Method	Route	Role	Meaning
GET	/api/quotations?status=&ownerId;=&customerId;=	Internal	List quotations with filters
POST	/api/quotations	SalesRep, Manager	Create draft quotation
GET	/api/quotations/{id}	Owner/Manager/Admin/Finance	Full quotation detail
PUT	/api/quotations/{id}	Owner before lock	Update customer/basic quote fields
POST	/api/quotations/{id}/lines	Owner	Add line and reprice
PUT	/api/quotations/{id}/lines/{lineId}	Owner	Change qty/discount/variant
DELETE	/api/quotations/{id}/lines/{lineId}	Owner	Remove line
POST	/api/quotations/{id}/recalculate	Owner/Manager	Recalculate price, margin, risk, approval route
GET	/api/quotations/{id}/recommendations	Owner	Return ranked upsell/cross-sell suggestions
POST	/api/quotations/{id}/recommendations/{productId}/accept	Owner	Add suggestion to quote
POST	/api/quotations/{id}/recommendations/{productId}/dismiss	Owner	Dismiss suggestion for this quote
POST	/api/quotations/{id}/submit	Owner	Lock quote and trigger approval/ready state
POST	/api/quotations/{id}/send-portal	Owner	Create customer portal access and mark Sent
GET	/api/quotations/{id}/audit	Owner/Manager/Admin/Finance	View quote audit trail

QuotationCreateRequest

```
customerId, currencyCode, priceListId?, expectedCloseDate?, notes?, lines[]
lines[] = productId, variantId?, quantity, discountPercent, subscriptionPlanId?
```

QuotationLineUpdateRequest

quantity, discountPercent, variantId?, subscriptionPlanId?. Unit price should **not** be trusted from the client; server selects the correct price list price.

QuotationDetailResponse

Include quotation header, customer/tier, lines with list/unit/net prices, discount, margin, totals, risk score, approval status, approval steps, recommendations, health, and current allowed actions.

11. Discount Governance & Blended Risk — Exact Server Logic

The supplied statement explains that discount limits can differ by customer tier and product category. A quote mixing categories must compute a blended risk and route to the highest required approval level. The source example: Gold allows 15%; Hardware allows 15%; Service allows only 10%; a Service line at 18% breaks its stricter limit and flags the entire quotation. ■

11.1 Per-line allowed discount

For each quotation line, determine **allowedPercent = min(customerTierCeiling, categoryCeiling)** when both exist. If a category rule is absent, fall back to customer-tier ceiling. The actual line discount is compared against that allowed value.

11.2 Over-limit points

For each line: **overage = max(0, actualDiscountPercent - allowedPercent)**. A line with 18% actual vs 10% allowed has 8 points over.

11.3 Blended score

A simple, explainable formula suitable for the hackathon is:
weightedOverage = Σ(lineNetBeforeDiscount × overagePoints) / Σ(lineNetBeforeDiscount).
Then map the score to configured approval bands. This captures the source requirement that several smaller violations across lines should add up rather than allowing each line to hide the overall margin loss.

Do not hardcode '8 points = Finance' in code. Configure approval bands in ApprovalRules. Example seed configuration: score 0–0 = no approval; >0–5 = Manager; >5–10 = Manager + Finance; >10 = Manager + Finance + critical flag.

11.4 Approval routing algorithm

1. Load customer tier and every quote line category.
2. Resolve each line's allowed discount.
3. Calculate overage and weighted blended risk score.
4. Find the highest configured approval level required by the score and any category-specific escalation.
5. Create approval request step 1 (Manager) if required.
6. Create Finance step only if the routing rule requires it.
7. Set quotation ApprovalStatus = Pending and Quotation Status = PendingApproval.
8. Create audit record explaining the score and reason.

11.5 Critical rule

The approval decision must be recalculated whenever quantity, product, category, price list, customer tier, or discount changes—and again after customer negotiation. Never let the frontend reuse a previous approval decision.

12. API Catalogue — Approval Workflow

Method	Route	Role	Purpose
GET	/api/approvals/pending	Manager/Finance	Current user's approval queue
GET	/api/approvals/{id}	Approver/Manager/Admin	Approval detail
POST	/api/approvals/{id}/approve	Assigned approver	Approve current step
POST	/api/approvals/{id}/reject	Assigned approver	Reject quote
POST	/api/approvals/{id}/return	Assigned approver	Return to rep for revision
GET	/api/quotations/{id}/approval-history	Internal authorized	Full approval timeline

ApprovalActionRequest

reason: required for reject/return; optional or short confirmation for approve. The server determines whether the caller is the correct role and whether this approval step is currently actionable.

Approval state machine

Draft → PendingApproval → ManagerApproved → FinanceApproved → Approved. Any reject → Rejected. Any return → RevisionRequired → rep edits → Recalculate → PendingApproval again.

If a quote is approved and then changed in a way that affects risk, the old approval is invalid. Create a new approval request chain rather than silently editing the old approval.

13. Upsell & Cross-Sell Logic

The source marks upsell/cross-sell rule setup as optional, but the frontend experience and key outcomes explicitly expect live suggestions while building the quote. Therefore, implement a simple deterministic recommendation engine rather than an AI model.

13.1 Ranking logic

For every product not already in the quote, find rules whose trigger product is present. Score candidates using: promoted +30; historical/co-purchase rule +20; margin above minimum +20; compatible category +10. Sort descending and return top 5.

13.2 Margin delta

For a suggested product, compute **suggestedMargin = (sellingPrice - costPrice) × quantity**. Show current quote margin and margin after addition. When accepted, the server adds the line, recalculates totals/risk, and returns the updated quote.

Endpoints

GET	/api/quotations/{id}/recommendations	Returns ranked suggestions with margin delta and promotion tag
POST	/api/quotations/{id}/recommendations/{productId}/accept	Adds suggestion as a quote line
POST	/api/quotations/{id}/recommendations/{productId}/dismiss	Records dismissal if desired

14. Multi-Warehouse Fulfillment & Backorders

The source requires automatic order splitting across warehouses based on stock, with manual override, shipment count/cost visibility, and a consolidation prompt if stock arrives later.

14.1 Recommended allocation algorithm

1. For each order line, calculate required quantity.
2. Load all active warehouses with available stock = OnHand – Reserved.
3. Prefer a warehouse that can satisfy the whole quantity; choose lowest shipping cost weight.
4. If no single warehouse can satisfy it, sort warehouses by available quantity descending, then shipping weight ascending.
5. Allocate quantities until required quantity is fulfilled.
6. Any remaining quantity becomes backorder quantity.
7. Return allocation preview before final confirmation.

14.2 API

Method	Route	Role	Meaning
GET	/api/orders/{id}/fulfillment-preview	Finance/Operations, Manager, Admin	Calculate recommended split without committing
POST	/api/orders/{id}/fulfillment/accept	Finance/Operations	Persist suggested allocations/reservations
PUT	/api/orders/{id}/fulfillment/override	Finance/Operations	Replace split manually
GET	/api/orders/{id}/backorders	Finance/Operations, Manager	View unfulfilled quantities
POST	/api/orders/{id}/backorders/consolidate	Finance/Operations	Use newly available stock to fulfill remaining quantities

14.3 Stock safety

Reservation and allocation must occur inside a transaction. Never allow reserved quantity to exceed OnHand. Use a concurrency strategy (rowversion recommended) so two orders cannot both reserve the same stock.

15. Hybrid Billing — One-Time + Recurring in One Order

A single order may contain hardware/one-time lines and recurring subscription lines. The system must display them separately and maintain recurring billing schedules.

15.1 Product types

ProductType	Meaning	Billing behavior
OneTime	Hardware/service sold once	Included in one-time invoice
Subscription	Recurring service/plan	Creates BillingSchedule; billed by plan frequency

15.2 Billing API

Method	Route	Role	Purpose
GET	/api/orders/{id}/billing	Finance/Operations, Manager, Admin	View invoice/schedule status
POST	/api/orders/{id}/billing/generate	Finance/Operations	Generate due invoice(s)
GET	/api/billing-schedules?status=&dueBefore;=	Finance/Operations	List recurring schedules
POST	/api/billing-schedules/{id}/generate-invoice	Finance/Operations	Generate recurring invoice
POST	/api/subscriptions/{id}/change	Finance/Operations	Change quantity/plan
POST	/api/subscriptions/{id}/cancel	Finance/Operations	Cancel recurring line
GET	/api/invoices/{id}	Finance/Operations, Manager, Admin, owner	Invoice detail
POST	/api/invoices/{id}/payments	Finance/Operations	Record payment
GET	/api/invoices/{id}/payments	Finance/Operations, Manager, Admin	Payment history
POST	/api/invoices/{id}/credit-notes	Finance/Operations	Create credit note for eligible refund

15.3 Proration formula

For a mid-cycle change, use a simple daily proration: **proratedAmount = priceDifference × remainingDays / totalDaysInCycle**. Store the resulting adjustment explicitly. If positive, add to next bill/current invoice; if negative and policy allows, create a credit note.

Do not silently mutate historical invoice lines. Historical invoices are immutable; changes produce new adjustment/credit records.

16. Customer Portal Negotiation — Separate Restricted Surface

The source explicitly requires a real, separate customer-facing negotiation screen, not merely an internal screen with a different label. This is a critical judging point.

16.1 Portal security design

- Customer logs in using email/password or a one-time magic link.
- Portal token resolves to CustomerId and permitted QuotationId(s).
- Portal API must reject any quotation not belonging to that customer.
- Do not expose internal approval notes, cost price, margin, warehouse stock, or other internal-only fields.
- Customer sees only commercial quotation data needed for negotiation.

16.2 Portal endpoints

Method	Route	Auth	Purpose
POST	/api/portal/auth/login	Portal credentials	Customer portal login
POST	/api/portal/auth/magic-link	Anonymous	Issue magic link if implemented
GET	/api/portal/quotations	Customer	List own active quotations
GET	/api/portal/quotations/{id}	Customer	View own quotation
POST	/api/portal/quotations/{id}/line-requests	Customer	Ask line-level question/change
POST	/api/portal/quotations/{id}/counter-discount	Customer	Submit counter discount
POST	/api/portal/quotations/{id}/confirm	Customer	Confirm final terms
GET	/api/portal/quotations/{id}/history	Customer	View customer-visible negotiation history

16.3 Negotiation logic

1. Customer submits a line change or counter discount.
2. Quotation moves to UnderNegotiation and records a QuotationChange.
3. Server applies the requested terms to a pending revision—not the final approved snapshot.
4. Discount/risk is recalculated.
5. If new risk requires approval, create a fresh approval chain and set PendingApproval.
6. If no approval is required, keep it in negotiation/ready state.
7. Customer sees only status and commercial result; internal approval reasons remain hidden.
8. Customer confirms only when the current revision is eligible.

17. Deal Health & Anomaly Engine

The source requires stalled quote detection, discount anomaly alerts, delivery promise slippage indicators, and an action/nudge/escalation from an alert.

17.1 Simple explainable rules

Signal	Rule	Severity
Stalled quote	No quote activity for configured N days while not terminal	At Risk/Critical
Discount anomaly	Current discount > rep historical average + configured margin	Warning/At Risk
Delivery slippage	Promised date < today and order not fulfilled	At Risk/Critical
Missing next action	Open quote has no planned activity/follow-up	Warning
Approval stuck	Approval pending beyond configured SLA	At Risk

17.2 Health score

Use a transparent score rather than an opaque ML model. Start at 100. Subtract configured penalties for stalled activity, overdue expected close date, stuck approval, missing next action, delivery slippage, and severe discount anomaly. Clamp 0–100. Label 70–100 Healthy, 40–69 At Risk, 0–39 Critical.

17.3 API

GET	/api/dashboard/deal-health	Manager, Admin, SalesRep	Summary cards and alerts
GET	/api/deal-health/alerts?severity=&type;=	Manager, Admin, SalesRep	Filtered alerts
GET	/api/quotations/{id}/health	Authorized internal	Detailed health explanation
POST	/api/deal-health/alerts/{id}/nudge	Manager, SalesRep	Create notification/action
POST	/api/deal-health/alerts/{id}/escalate	Manager, Admin	Escalate responsible user/team

18. Order, Invoice, Payment APIs

Method	Route	Role	Purpose
POST	/api/quotations/{id}/confirm-order	SalesRep/Manager/Finance as allowed	Convert eligible quotation into order
GET	/api/orders	Internal	List orders
GET	/api/orders/{id}	Internal authorized	Order detail
GET	/api/orders/{id}/lines	Internal authorized	Order lines
POST	/api/orders/{id}/reserve-stock	FinanceOperations	Reserve stock based on allocation
GET	/api/invoices	FinanceOperations, Manager, Admin	Invoice list
GET	/api/invoices/{id}	FinanceOperations, Manager, Admin	Invoice detail
POST	/api/invoices/{id}/payments	FinanceOperations	Record payment
GET	/api/invoices/{id}/payments	FinanceOperations, Manager, Admin	Payment list
POST	/api/invoices/{id}/credit-notes	FinanceOperations	Credit note

Order confirmation guard

- Quotation must be in Approved or NegotiationConfirmed-with-no-approval-required state.
- All required approval steps must be completed.
- Customer must have confirmed if the business flow requires customer confirmation.
- Quote must contain at least one line.
- Server rechecks price/risk before conversion.
- Create Order and OrderLines as immutable commercial snapshots.

Payment status calculation

Outstanding = InvoiceTotal – Sum(valid payments). If outstanding <= 0 → Paid. If payments > 0 and outstanding > 0 → PartiallyPaid. If no payment and due date passed → Overdue. Otherwise → Unpaid/Pending.

19. Reporting & Dashboard APIs

The source explicitly requires filters for Period, Sales Team/Rep, Approval Status, Product/Category, plus PDF/XLS export.

Method	Route	Role	Filters
GET	/api/reports/sales-summary	Manager, Admin	from,to,teamId,repId
GET	/api/reports/quotations	Manager, Admin	from,to,teamId,repId,approvalStatus,categoryId
GET	/api/reports/products	Manager, Admin	from,to,categoryId,productId
GET	/api/reports/discounts	Manager, Admin	from,to,repId,categoryId
GET	/api/reports/fulfillment	Manager, Finance, Admin	from,to,warehouseId
GET	/api/reports/billing	Finance, Manager, Admin	from,to,status
GET	/api/reports/export/pdf	Manager, Admin	same filters
GET	/api/reports/export/xls	Manager, Admin	same filters

Recommended dashboard cards

- Open quotation value.
- Approved / pending / rejected quote counts.
- Weighted/at-risk pipeline.
- Discount anomaly count.
- Stalled quote count.
- Orders awaiting fulfillment.
- Backorder quantity/value.
- Outstanding invoice amount and overdue count.
- Recurring revenue scheduled (optional but useful).

20. DTO Catalogue — Naming and Responsibility

Use separate Request and Response DTOs. Avoid generic 'CreateModel' names that hide intent. DTO names should tell the developer what the endpoint expects.

Area	Request DTOs	Response DTOs
Auth	SignupRequest, LoginRequest	AuthResponse, MeResponse
Customer	CreateCustomerRequest, UpdateCustomerRequest	CustomerListItemResponse, CustomerDetailResponse
Product	CreateProductRequest, UpdateProductRequest, CreateVariantRequest	ProductListItemResponse, ProductDetailResponse
Price	CreatePriceListRequest, UpsertPriceListItemRequest	PriceListResponse
Discount	CreateDiscountRuleRequest, UpdateDiscountRuleRequest	DiscountRuleResponse
Approval	ApprovalActionRequest	ApprovalQueueItemResponse, ApprovalDetailResponse
Quote	CreateQuotationRequest, UpdateQuotationRequest, AddQuotationLineRequest, UpdateQuotationLineRequest	QuotationListItemResponse, QuotationDetailResponse
Negotiation	LineChangeRequest, CounterDiscountRequest	PortalQuotationResponse, NegotiationHistoryItemResponse
Fulfillment	FulfillmentOverrideRequest	FulfillmentPreviewResponse, AllocationResponse
Billing	SubscriptionChangeRequest, PaymentRequest, CreditNoteRequest	InvoiceResponse, BillingScheduleResponse
Dashboard	DashboardFilterRequest (or query params)	DashboardResponse, DealHealthAlertResponse

Rule: if a DTO is used by both admin and customer portal, create a safe portal-specific response DTO rather than accidentally leaking internal fields such as CostPrice, MarginAmount, RiskFormulaDetails, or approval comments.

21. Validation Matrix

Object	Validation
Customer	Name required; valid email if supplied; tier required; currency valid
Product	Name/SKU/category/type required; BasePrice >= 0; CostPrice >= 0; TaxRate 0..100
PriceListItem	Product exists; price >= 0; unique price-list/product/currency
Quotation	Customer exists; at least one line before submit; currency matches pricing rule
QuotationLine	Product active; quantity > 0; discount 0..100; subscription plan required for recurring product
Discount	Server resolves price; client cannot force unit price; discount checked against rules
Approval	Only current step assignee can act; rejected/approved steps cannot be acted on twice
Warehouse	Warehouse active; quantity available; override cannot exceed available unless intentionally backordered
Subscription	Plan compatible with product; dates valid; quantity > 0
Payment	Invoice exists; amount > 0; cumulative payment <= invoice total
Portal	Customer can access only their own quotation; quote must be portal-visible

ASP.NET implementation

Use FluentValidation or built-in DataAnnotations for DTO shape validation, plus domain/service validation for business rules. Example: FluentValidation checks discountPercent between 0 and 100; DiscountService decides whether 12% is permitted for this customer/category. Do not put business rules into controllers.

22. State Machines — Never Allow Random Status Changes

Quotation states

Draft → PendingApproval → Approved → Sent → UnderNegotiation → Confirmed → ConvertedToOrder. Alternative terminal states: Rejected, Cancelled. A negotiation change may return the quote to PendingApproval.

Approval states

Pending → Approved OR Rejected OR Returned. Returned creates a revision path; after the rep edits, a new approval request is created.

Order states

Confirmed → PartiallyAllocated → Allocated → PartiallyFulfilled → Fulfilled. Backorder can coexist with PartiallyFulfilled until consolidated.

Invoice states

Draft → Issued → PartiallyPaid → Paid; Issued → Overdue when due date passes without full payment; credit note/void should be separate actions rather than arbitrary status edits.

Subscription states

Active → Modified → Active, or Active → Cancelled. Billing schedule rows should preserve historical billed periods.

Implementation rule

Expose explicit service methods such as SubmitQuotationAsync(), ApproveQuotationAsync(), RejectQuotationAsync(), ConfirmOrderAsync(), AllocateFulfillmentAsync(), RecordPaymentAsync(). Avoid generic UpdateStatusAsync() because it bypasses business invariants.

23. React Frontend Route & Component Plan

Route	Role	Purpose
/login	Anonymous	Internal login
/signup	Anonymous	Internal SalesRep signup
/dashboard	Internal	Role-aware dashboard
/workspace/quotations	SalesRep/Manager	Quotation list
/workspace/pipeline	SalesRep/Manager	Kanban pipeline
/workspace/quotations/:id	SalesRep/Manager	Quotation builder
/workspace/quotations/:id/approval	Manager/Finance	Approval detail
/workspace/quotations/:id/fulfillment	Finance/Manager	Warehouse split
/workspace/quotations/:id/billing	Finance/Manager	Billing
/portal/login	Customer	Separate portal login
/portal/quotations	Customer	Own quotation list
/portal/quotations/:id	Customer	Negotiation/confirmation
/admin/products	Admin	Product/category setup
/admin/pricing	Admin	Price lists
/admin/discounts	Admin/Manager	Discount tiers/rules
/admin/approvals	Admin/Manager	Approval chain
/admin/warehouses	Admin/Finance	Warehouse + stock
/admin/subscriptions	Admin	Subscription plans
/reports	Manager/Admin/Finance	Reports

23.1 Frontend API rule

Create one API client (axios/fetch wrapper) that automatically sends JWT and normalizes errors. Then create domain clients such as authApi, quotationApi, approvalApi, fulfillmentApi, billingApi, portalApi. Do not scatter raw fetch URLs across components.

23.2 State management

For a 24-hour project, React Query/TanStack Query is ideal for server state; local component state is enough for the quote cart draft. If avoiding an additional library, use Context for auth and a small API service layer.

24. Complete Endpoint-by-Role Matrix

Role	Must-have endpoints
Admin	All /auth/me; user CRUD; customer tiers; categories; products; variants; price lists; discount rules; approval rules; warehouses/stock; subscription plans; upsell rules; reports; deal health
SalesRep	/auth/me; customers search/create; products/search; quotations CRUD/lines/recalculate/recommendations/submit/send; own pipeline; own audit; own health; portal negotiation responses
SalesManager	All SalesRep read/management where team-scoped; pending approvals; approve/reject/return; discount/approval configuration; deal health; reports
FinanceOperations	Products/customers read; approval second level; fulfillment preview/accept/override; backorders; billing schedules; invoices; payments; credit notes; operational reports
Customer	Only /api/portal/* routes for own quotations; never internal /api/quotations/* or inventory/margin/approval internals

Authorization should be tested at both controller and service scope. Example: a SalesRep cannot simply change a URL from /quotations/101 to /quotations/102 and access another rep's quote.

25. The Four Most Important Service Flows

Flow A — Quote pricing and discount

1. QuotationService loads customer tier and price list.
2. For each line, PriceListService resolves authoritative unit price.
3. DiscountService resolves customer/category ceiling and calculates line overage.
4. DiscountService calculates blended risk score.
5. ApprovalService resolves approval chain.
6. QuotationService calculates subtotal, discount total, tax, grand total, cost total, margin amount and margin percent.
7. AuditService records recalculation/important changes.
8. Response returns all values needed by React.

Flow B — Approval

1. Rep submits quote.
2. Service locks the editable commercial fields.
3. If risk score requires no approval, status becomes Approved/Ready.
4. Otherwise create Manager step; if required create Finance step.
5. Manager acts. If rejected, quote becomes Rejected. If returned, RevisionRequired.
6. If Manager approves and Finance is required, Finance step becomes current.
7. When all required steps approve, quote becomes Approved.

Flow C — Customer negotiation

1. Portal customer submits counter/change request.
2. Create QuotationChange and mark UnderNegotiation.
3. Apply requested revision to a controlled draft revision.
4. Recalculate discount/risk.
5. If approval needed, create fresh approval chain.
6. If approved/no approval needed, customer can confirm.
7. Confirm converts eligible quote to order.

Flow D — Fulfillment + billing

1. Order confirmation creates order lines as snapshots.
2. FulfillmentService calculates warehouse split.
3. Accept/override reserves stock and records allocations.
4. Unfulfilled quantities create backorders.
5. One-time lines create invoice lines.
6. Recurring lines create BillingSchedule rows.
7. Due recurring schedules generate recurring invoices.
8. Payments update invoice status.

26. Controller / Service / DTO Pattern

Controller example (conceptual)

```
[ApiController] [Route("api/quotations")] [Authorize(Roles = "SalesRep,SalesManager,Admin")]
public class QuotationsController : ControllerBase { [HttpPost("{id}/submit")] public async
Task> Submit( int id, CancellationToken ct) => Ok(await _quotationService.SubmitAsync(id,
User.GetUserId(), ct)); }
```

Service example (conceptual)

```
public async Task SubmitAsync( int quotationId, int userId, CancellationToken ct) { var quote =
await LoadQuoteForUpdateAsync(quotationId, userId, ct); ValidateEditableOwner(quote, userId);
var pricing = await _pricingService.RecalculateAsync(quote, ct); var route = await
_approvalService.ResolveRouteAsync(pricing, ct); quote.Status = route.RequiresApproval ?
QuotationStatus.PendingApproval : QuotationStatus.Approved; await
_auditService.RecordAsync(...); await _db.SaveChangesAsync(ct); return MapToResponse(quote); }
```

The actual implementation should be richer, but the architecture is the important part: Controller = HTTP; Service = business decision; EF Core = persistence; DTO = API contract.

27. Reliability: Errors, Audit, Concurrency

Global exception middleware

- ValidationException → 400/422 with field errors.
- UnauthorizedAccessException → 403.
- KeyNotFoundException → 404.
- BusinessRuleException → 400/409 with stable error code.
- DbUpdateConcurrencyException → 409 'record changed; reload'.
- Unexpected exception → 500 with traceId; log full server-side details only.

Audit requirements

The source explicitly requires approvals, rejections, and edits to be logged with user, timestamp, and reason. Extend that to all high-risk commands: discount changes, approval decisions, negotiation changes, warehouse overrides, stock adjustments, billing changes, and payments.

Recommended audit actions: QUOTE_CREATED, LINE_ADDED, LINE_CHANGED, DISCOUNT_CHANGED, RISK_RECALCULATED, APPROVAL_REQUESTED, APPROVED, REJECTED, RETURNED, NEGOTIATION_REQUESTED, QUOTE_CONFIRMED, ORDER_CREATED, STOCK_RESERVED, FULFILLMENT_OVERRIDDEN, BACKORDER_CREATED, BILLING_GENERATED, PAYMENT_RECORDED, CREDIT_NOTE_CREATED.

Concurrency

Add SQL Server rowversion/byte[] concurrency tokens to Quotations, InventoryStocks, Orders, and Invoices. The frontend sends the latest version or the API uses an If-Match-style mechanism. This prevents stale screens from overwriting newer approval/stock/payment changes.

28. Testing Strategy — Before Demo

The source supplies an eight-step quick test flow. Your test suite should reproduce those steps plus authorization and edge cases. The source says each step should produce a visible, correct result before moving to the next.

28.1 Must-pass end-to-end test

1. Create/log in SalesRep.
2. Admin seed/configure Bronze/Silver/Gold, warehouse(s), subscription plan, product categories and discount rules.
3. Create a Gold customer.
4. Create quote with Hardware at 12% and Service at 18% where Service ceiling is 10%.
5. Confirm blended risk > 0 and Manager approval is automatically created.
6. Add one upsell suggestion; confirm total and margin change immediately.
7. Manager approves; if risk band requires Finance, verify Finance step appears and Manager alone cannot finish the quote.
8. Generate fulfillment preview; verify split across two warehouses if one lacks stock.
9. Confirm one-time + recurring lines create one-time invoice lines and a recurring schedule.
10. Open customer portal as customer; submit higher counter discount.
11. Verify quote re-enters approval automatically.
12. Approve again; customer confirms.
13. Confirm order, reserve stock, create invoice, record payment.
14. Verify invoice becomes PartiallyPaid/Paid correctly.
15. Open deal health dashboard and verify stalled/anomaly/delivery alerts.

28.2 Authorization test matrix

Test	Expected
SalesRep calls Admin product POST	403
SalesRep opens another rep's quote	404 or 403; do not leak existence
Customer calls internal quotation endpoint	401/403
Customer opens another customer's portal quote	404/403
Manager approves Finance-only step	403
Finance edits discount rules without permission	403
Inactive user logs in	401/403

28.3 Business edge cases

- No stock anywhere → full backorder.
- Exact stock → one warehouse, no split.
- Stock split → two or more allocations.
- Discount exactly at ceiling → no violation.
- Discount 0% → no approval from discount governance.
- Several small overages → blended score still increases.
- Quote approved, then discount changed → approval invalidated and recalculated.
- Customer counter discount below/at ceiling → no approval if rules allow.
- Payment equal to outstanding → Paid.
- Payment less than outstanding → PartiallyPaid.

- Overpayment attempt → rejected.
- Subscription cancellation mid-cycle → proration/credit rule applied.

29. Seed Data — Make the Demo Deterministic

A 24-hour demo should never depend on typing 30 configuration records live. Seed the database with a coherent scenario.

Seed group	Suggested records
Roles	Admin, SalesRep, SalesManager, FinanceOperations, Customer
Customer tiers	Bronze 5%, Silver 10%, Gold 15%
Categories	Hardware, Services, Subscriptions
Products	Laptop, Dock, Setup Service, Premium Support Subscription
Price list	Standard INR price list with all demo products
Discount rules	Gold Hardware 15%; Gold Services 10%; Gold Subscription 12%
Approval rules	Risk 0 = none; low risk = Manager; higher risk = Manager + Finance
Warehouses	Main Warehouse, East Depot
Stock	Main: Laptop 5, Dock 10; East: Laptop 8, Dock 5; Service unlimited/non-stock
Subscription plans	Monthly, Quarterly, Yearly
Upsell rules	Laptop → Dock; Laptop → Premium Support
Users	admin@demo, rep@demo, manager@demo, finance@demo
Customer	Acme Corp, Gold tier, portal user

Demo scenario

Quote: 2 Laptops (12% discount) + Setup Service (18% discount) + Premium Support monthly subscription. This intentionally exercises category-specific discount governance, upsell, warehouse splitting, hybrid billing, customer negotiation, re-approval, and payment.

30. 24-Hour Build Plan

Time	Goal	Deliverable
Hour 0–1	Architecture + DB	Solution, DbContext, entities/enums, migrations
Hour 1–3	Auth/Roles	JWT, signup/login, role authorization, seed users
Hour 3–5	Admin config	Products, categories, tiers, price lists, discount/approval rules
Hour 5–8	Quotation core	Quote CRUD, lines, price calculation, totals, margin
Hour 8–10	Discount/approval	Blended risk, automatic routing, approve/reject/return, audit
Hour 10–12	React workspace	Quote builder, product picker, totals, approval status
Hour 12–14	Fulfillment	Warehouse stock, split preview, accept/override, backorder
Hour 14–16	Billing	One-time invoice + recurring schedule + payment
Hour 16–18	Portal	Separate customer login/view/negotiation/confirm
Hour 18–20	Health/reporting	Stalled/anomaly/delivery alerts + dashboard filters
Hour 20–22	Integration	Frontend/backend all flows, error handling, seed reset
Hour 22–23	Testing	Eight-step source flow + role/edge tests
Hour 23–24	Demo polish	Architecture page, README, screenshots, 5-minute demo rehearsal

Priority rule

Do not start with AI, charts, exports, or visual polish. First make the server correctly complete Quote → Approval → Fulfillment → Billing → Payment. Then add portal negotiation and health. Finally add optional polish.

31. Implementation Order in Code

1. Create solution/projects and install packages.
2. Create enums first: Role, ProductType, QuoteStatus, ApprovalStatus, ApprovalLevel, OrderStatus, InvoiceStatus, SubscriptionStatus.
3. Create EF entities and DbContext.
4. Configure relationships/indexes/decimal precision/rowversion.
5. Create first migration and database.
6. Seed roles + demo configuration + users.
7. Implement auth middleware/JWT.
8. Implement configuration services/controllers.
9. Implement quotation pricing service.
10. Implement discount/risk service.
11. Implement approval service.
12. Implement quote APIs and integration tests.
13. Implement recommendation service.
14. Implement order conversion + fulfillment.
15. Implement billing/payment.
16. Implement portal auth and negotiation.
17. Implement deal health/reporting.
18. Build React pages in the same order as the APIs.
19. Run source eight-step test from a clean seeded database.

31.1 Package shortlist

Backend	Purpose
ASP.NET Core Web API	HTTP API
EF Core + SQL Server provider	Code First persistence
JWT Bearer authentication	Internal/portal authentication
BCrypt/Argon2 password hashing library	Password hashing; use a maintained library
FluentValidation	DTO validation
Swagger/OpenAPI	API testing/documentation
Serilog (optional)	Structured logging
ClosedXML (optional)	XLS export
QuestPDF (optional)	PDF report export
Frontend	**Purpose**
React + Vite	Application UI
React Router	Role-aware routing
Axios or fetch wrapper	API client
TanStack Query (optional)	Server-state caching
Tailwind CSS / component library	Fast polished UI

32. Endpoint Count and Scope

The exact count is an engineering choice. The blueprint above contains approximately 100 endpoints if every configuration, report, portal, and operational action is implemented separately. For a 24-hour hackathon, the goal is not to maximize endpoint count; it is to ensure every endpoint has a single clear meaning and every important business rule has a server-side owner.

Module	Approx. endpoints	Priority
Auth/User	10	Must
Configuration	35+	Must for core; optional endpoints can be reduced
Customers/Pipeline	10	Must
Quotation/Recommendations	15	Must
Approval	6	Must
Fulfillment	5	Must
Billing/Payment	10	Must
Portal	7	Must
Health	5	Strongly recommended
Reports	8	Recommended

If time is tight, combine read endpoints rather than deleting business logic. For example, one **GET /api/quotations/{id}** can return quote + approval + health + recommendations summary. But do not combine unrelated write operations into one giant endpoint.

33. Common Mistakes That Would Weaken the Submission

- Hardcoding approval results in React or setting 'approved=true' for the demo.
- Calculating discount risk only from the average discount and ignoring category ceilings.
- Allowing the frontend to submit arbitrary unit prices.
- Using a fake warehouse split that does not inspect stock.
- Showing a customer portal that can see margin/cost/approval internals.
- Changing an approved quote without invalidating the approval.
- Mixing recurring and one-time billing into one unexplained total.
- Allowing overpayment or negative payment.
- Using generic UpdateStatus endpoint that bypasses state transitions.
- Building 20 screens with no end-to-end integration.
- Starting with AI before the transactional flow works.
- Using mock JSON in the frontend after the backend is supposedly complete.
- No audit trail for discount/approval/negotiation changes.
- No seeded scenario, making the five-minute demo fragile.

The source specifically emphasizes that approval routing, discount governance, warehouse splitting, and billing proration must be implemented in application logic, not hardcoded or faked for the demo. It also explicitly requires the customer negotiation view to be a real separate restricted view.

34. Five-Minute Judge Demo Script

Time	Action	What judge should see
0:00–0:30	Login as Rep	Role-aware workspace
0:30–1:15	Create quote + add products	Live pricing, margin, upsell
1:15–1:45	Apply high service discount	Risk score changes; approval auto-created
1:45–2:15	Manager approves	Audit trail + status transition
2:15–2:45	Fulfillment preview	Two warehouses + quantities + shipment cost
2:45–3:15	Show hybrid billing	One-time invoice + recurring schedule
3:15–4:00	Customer portal counter discount	Separate portal; re-approval triggered
4:00–4:30	Confirm + payment	Order/invoice/payment status updates
4:30–5:00	Deal health/reporting	Stalled/anomaly/fulfillment indicators + filters

The demo should narrate business value, not code. Example: 'The rep tried to discount a thin-margin service beyond its category ceiling. The backend detected the violation, calculated the blended risk, and automatically routed the quote to the manager. The customer later negotiated an even larger discount, so the same engine invalidated the old approval and restarted the correct chain.'

35. Final Definition of Done

Area	Done when
Authentication	Login/signup works; JWT; roles; inactive user blocked
Admin config	Products, price lists, tiers, discount/approval rules, warehouses, plans configured
Quote	Create/edit lines; server pricing; totals/margin; audit
Discount governance	Per-line category/tier ceiling + blended risk + automatic approval routing
Approval	Manager/Finance sequence; approve/reject/return; audit; re-entry
Upsell	Suggestions are data-driven; accept updates margin immediately
Fulfillment	Stock-aware split; manual override; reservation; backorder
Hybrid billing	One-time invoice lines + recurring schedule; proration/credit path
Portal	Separate restricted customer view; line requests/counter; confirm
Re-approval	Negotiation terms that worsen risk automatically restart approval
Deal health	Stalled, discount anomaly, delivery slippage; actionable alert
Reports	Period/team/rep/status/product filters; export if time
Testing	Source eight-step flow passes from clean seed
Demo	Two end-to-end flows can be shown in five minutes
Documentation	One-page architecture diagram + next-step note

36. Final Engineering Rule

If a business rule can change the money, approval, inventory, customer commitment, or billing result, it belongs in the backend service layer and must be testable independently. React is the experience layer; ASP.NET is the business authority; SQL Server is the durable source of truth.

Source alignment notes

- Pages 1–3: problem definition, goals, roles.
- Pages 4–8: backend configuration and frontend modules.
- Page 9: end-to-end flow.
- Page 10: technical guidelines and deliverables.
- Page 11: quick test flow and blended-risk introduction.
- Page 12: blended-risk example and rationale.
- Page 13: business importance and production-style expectations.

This document intentionally translates the supplied challenge into a concrete ASP.NET/SQL Server/React implementation plan. Where the source leaves implementation choices open (for example exact API names, exact database schema, exact risk formula, or exact JWT mechanism), the choices in this document are explicit engineering recommendations rather than claims that the source mandated those exact names or formulas.